BusGuard AI: Predictive Bus Delay & Automated Recovery System

BusGuard AI is an operational decision-support and closed-loop mitigation system designed for urban transit operations (specifically tailored for Rapid KL / Klang Valley corridors). Rather than merely reporting delays after buses are already bunched or stranded, BusGuard AI detects emerging bottlenecks, evaluates feasible fleet interventions (e.g., dynamic standby-bus injection, departure retiming, headway smoothing), enforces safety and confidence gates, and executes simulated interventions with real-time impact verification.

Table of Contents

- Key Capabilities & The 4 Pillars
- System Architecture
- Repository Structure
- Installation & Environment Setup
- Running the Application
- Running the Test Suite
- Data Requirements & Schema
- Recovery Workflow & Safety Policies
- Limitations & Simulation Assumptions
- License

Key Capabilities & The 4 Pillars

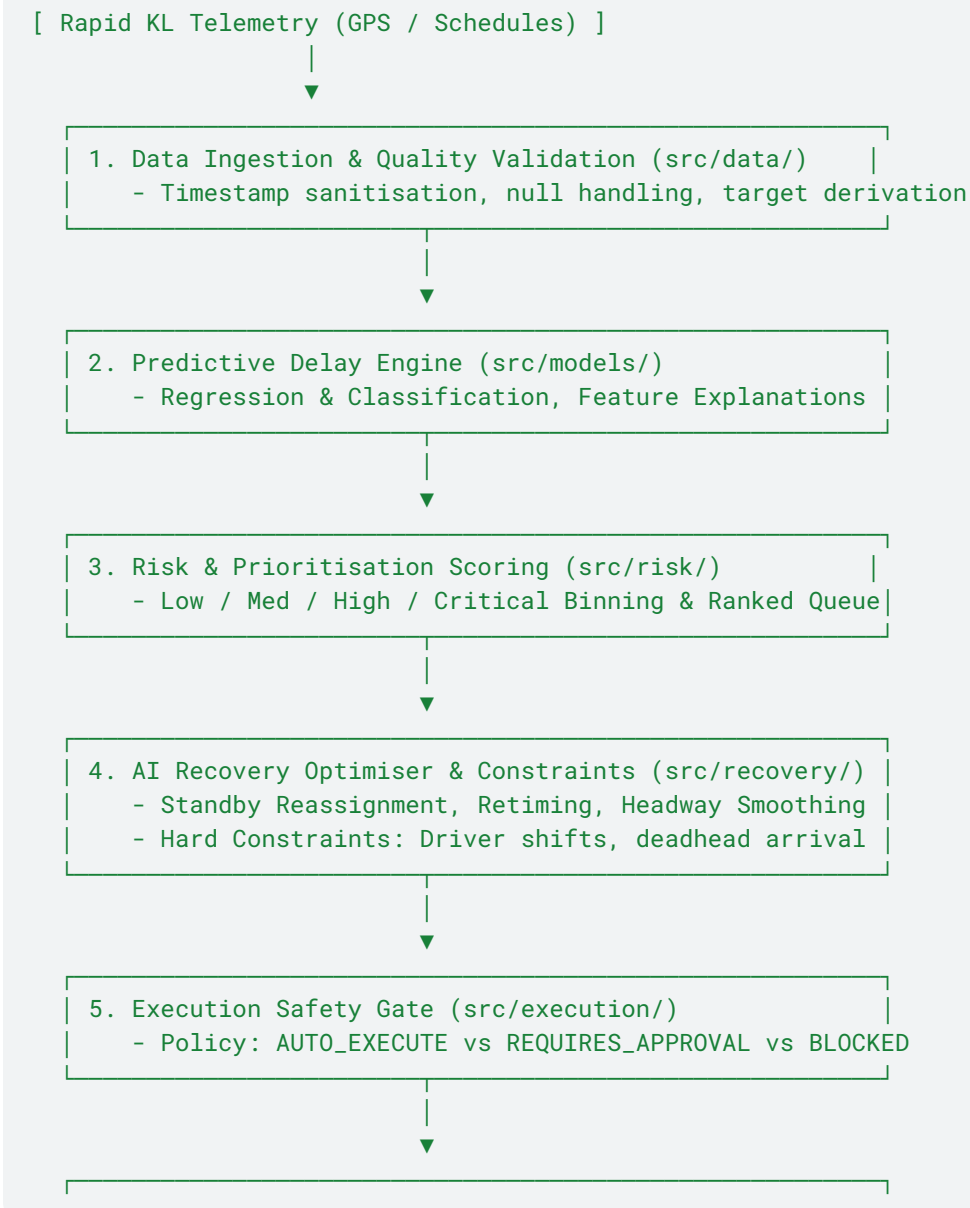
BusGuard AI is built around four fundamental operational pillars:

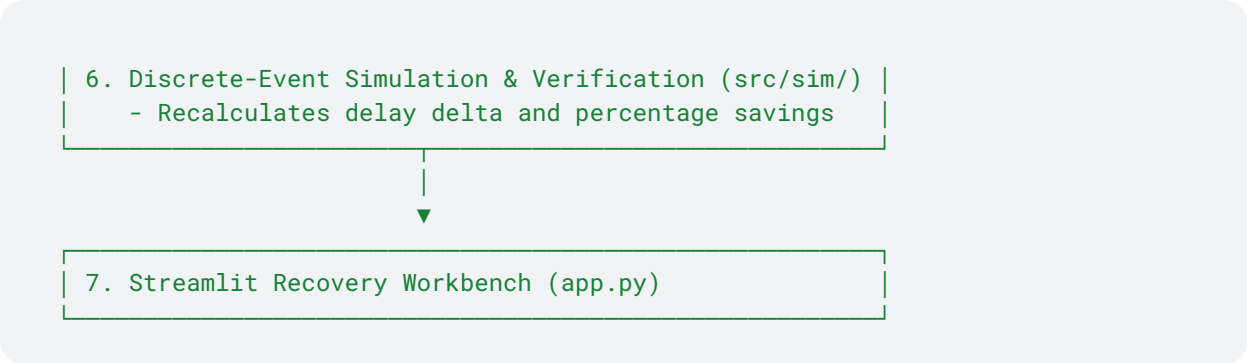
1. **DETECT**: Ingests vehicle GPS telemetry and scheduled timetables to predict downstream delay duration ($\hat{d}_i$) and delay escalation probability (P_i) using trained machine learning models (Random Forest / XGBoost).
2. **CATEGORISE**: Classifies trips into operational risk tiers (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`) and identifies likely underlying bottlenecks (e.g., corridor congestion, terminal turnaround deficit).

3. **PRIORITISE**: Ranks trips through an operational multi-factor scoring formula that balances delay magnitude, escalation probability, time-to-impact urgency, and passenger load indicators.
4. **EXPLAIN**: Provides feature attributions (SHAP/feature importances) for predictive alerts, explains why the recovery optimiser selected a specific intervention over rejected alternatives, and proves simulated savings.

System Architecture

None





Repository Structure

None

```
busguard-ai/
├── app.py                          # Streamlit Recovery Workbench entry point
├── config/
│   ├── thresholds.yaml             # Risk thresholds, weights, and safety limits
│   └── routes_metadata.json        # Route geometry and bus stop identifiers
├── data/
│   ├── raw/                        # Telemetry input data (CSV/JSON)
│   ├── fixtures/                   # Deterministic test fixtures for pytest
│   └── synthetic/                  # Fallback simulation fixtures
├── models/
│   └── delay_predictor.joblib      # Serialised trained model artifact
├── src/
│   ├── data/
│   │   ├── loader.py               # Ingestion and schema validation
│   │   └── preprocessor.py         # Feature engineering & temporal split
│   ├── models/
│   │   ├── train.py                # Model training script
│   │   ├── predict.py              # Inference pipeline
│   │   └── explain.py              # Local feature contribution engine
│   ├── risk/
│   │   ├── categorizer.py          # Severity and risk binning
│   │   └── ranker.py               # Priority score ranking queue
│   ├── recovery/
│   │   ├── generator.py            # Candidate intervention generator
│   │   ├── constraints.py          # Hard constraint validators
│   │   └── optimizer.py            # Objective function maximiser
│   ├── execution/
│   │   ├── policy.py               # Safety gate (AUTO / APPROVAL / BLOCKED)
│   │   └── handler.py              # State mutation dispatcher
│   ├── simulation/
```

```
|   |   └── engine.py              # Post-action delta simulator
|   ├── verification/
|   |   └── metrics.py             # Impact metrics & zero-division checks
|   └── audit/
|       └── logger.py              # Append-only transaction audit ledger
├── tests/
|   ├── test_data_pipeline.py      # Schema & data sanitisation tests
|   ├── test_models.py             # Model inference & evaluation tests
|   ├── test_optimizer.py          # Constraint checking & recovery tests
|   ├── test_execution_safety.py   # Autonomous gate policy tests
|   └── test_verification.py       # Math integrity & precision tests
├── requirements.txt               # Pinned dependencies
├── PRD.md                         # Product Requirements Document
└── README.md                      # Project documentation (this file)
```

Installation & Environment Setup

Prerequisites

- Python 3.10, 3.11, or 3.12
- Git
- Virtual environment tool (`venv` or `conda`)

1. Clone the Repository

Shell

```
git clone https://github.com/your-team/busguard-ai.git
cd busguard-ai
```

2. Set Up Virtual Environment

Shell

```
# Using venv
python3 -m venv .venv

# Activate on Linux/macOS:
source .venv/bin/activate
```

```
# Activate on Windows (PowerShell):
.venv\Scripts\Activate.ps1
```

3. Install Dependencies

```Shell
pip install --upgrade pip
pip install -r requirements.txt
```

Running the Application

Launch the Streamlit Recovery Workbench

```Shell
streamlit run app.py
```

Once started, open your web browser at `http://localhost:8501`.

Pre-training / Re-training the Predictive Model (Optional)

If updating telemetry records or retraining the model:

```Shell
python -m src.models.train --data data/raw/rapidkl_telemetry.csv --output
models/delay_predictor.joblib
```

Running the Test Suite

The project includes unit and integration tests covering the end-to-end data processing, model evaluation, constraint validation, and impact calculations:

```shell
# Run all unit and integration tests
pytest tests/ -v

# Run with test coverage report
pytest --cov=src tests/ --cov-report=term-missing
```

Data Requirements & Schema

BusGuard AI processes tabular transit telematics matching standard GTFS-RT conventions:

Column Name	Type	Description
`trip_id`	String	Unique scheduled trip identifier
`route_id`	String	Rapid KL route identifier (e.g., `750`, `BET3`)
`bus_id`	String	Physical bus vehicle registration
`stop_id`	String	Checkpoint stop identifier
`scheduled_time`	ISO 8601	Timetable expected arrival timestamp
`observed_time`	ISO 8601	Actual telemetric checkpoint arrival timestamp
`latitude`	Float	WGS84 GPS latitude coordinate
`longitude`	Float	WGS84 GPS longitude coordinate
`speed_kmh`	Float	Current speed in km/h
`congestion_index`	Float	Corridor traffic index between `0.0` and `1.0`

If telemetry fields are incomplete, fallback values or test fixtures located in `data/fixtures/` are used and clearly tagged on the UI.

Recovery Workflow & Safety Policies

BusGuard AI implements a strict safety framework governing automated interventions:

```
None
+-------------------------------------------------------------------------------
--+
|                           EXECUTION DECISION MATRIX
|
+-------------------+-----------------+-------------------+-----------------------
--+
| Model Confidence  | Risk Severity   | Hard Constraints  | Outcome
|
+-------------------+-----------------+-------------------+-----------------------
--+
| >= 85%            | Low             | All Validated     | AUTO_EXECUTE
|
| >= 85%            | Medium / High   | All Validated     | REQUIRES_APPROVAL
|
| < 85%             | Any             | All Validated     | REQUIRES_APPROVAL
|
| Any               | Any             | Any Violated      | BLOCKED (Keep
Baseline) |
+-------------------+-----------------+-------------------+-----------------------
--+
```

- **`AUTO_EXECUTE`**: High-confidence, low-disruption actions applied directly to the simulated schedule.
- **`REQUIRES_APPROVAL`**: Queued on the human controller workbench for explicit approval or rejection.
- **`BLOCKED`**: Infeasible interventions (e.g., deadhead arrival exceeds delay horizon, driver shift breach) are rejected, maintaining the baseline timetable.

Limitations & Simulation Assumptions

1. **Simulation Sandbox**: Interventions are dispatched exclusively in a discrete-event simulation sandbox. The prototype does not issue live dispatch commands to Rapid KL physical road assets.
2. **Deadhead Travel Assumptions**: Standby bus transfer speeds are estimated using corridor historical averages.

3. **Cost Attribution**: Fuel and operator costs are computed from standard public transit operational estimates (e.g., RM 2.20/km) and marked with `[SIMULATED COST]`.
4. **Data Integrity**: Missing values are not fabricated. When telemetry is insufficient, predictions are explicitly flagged with `DATA_UNRELIABLE`.
